

Maltese Snow War

Hold, Dodge, and Throw!

Code structure, runtime flow, and P2P smoothness

A browser remake of the 1998 Flash game SnowCraft, with moonlab's Line Puppy dogs: Maltese vs golden retrievers. Vs AI (Easy / Hard) and 1v1 WebRTC rooms.

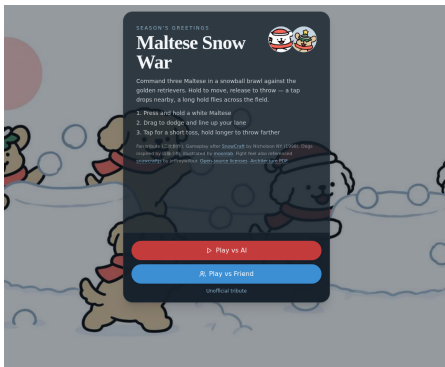
This note maps the repo, the fixed-step game loop, and every trick used to hide network delay.

1. What this is

Hold, Dodge, and Throw!

Maltese Snow War is a canvas snowball fight in the browser. Hold a dog to move, release to throw, pack snow between shots. Vs AI still uses tap-near / hold-far. Versus uses auto-aim plus a fixed range and speed. White Maltese (red scarves) stand on the right; brown retrievers (party hats) on the left. The host always plays the Maltese. The guest is mirrored and plays the retrievers. Two-hit bury, ellipse forts, pack-snow cooldown, and a 3-2-1 countdown before each heat. It is an unofficial fan tribute to Nicholson NY's *SnowCraft* (1998) and moonlab's Line Puppy illustrations.

Two modes: **Play vs AI** (Easy / Hard) on one machine, and **Play vs Friend** via a 6-letter room code. The host is always the white Maltese (right side). The guest's canvas is mirrored so they play the brown retrievers from the right-hand side of their screen. There is no dedicated game-simulation server — the host's browser is the authority; a small signaling server only helps the two browsers find each other.



Title card (below, not as a cover overlay): Play vs AI / Play vs Friend.



Host fight (no countdown overlay): brown retrievers left, white Maltese right. HUD: Maltese.

2. Stack

Layer	Choice	Why
UI shell	React 19 + TanStack Router + Tailwind	Title, lobby, pause, credits. Canvas sits underneath.
Game	TypeScript, Canvas 2D, fixed 1/60 s step	Deterministic-enough sim; render is a view, not the clock.
Net	WebRTC data channels + HTTP/SSE fallback	Peer-to-peer after join. TURN only if NAT blocks UDP.
Host	Vite + Nitro (Vercel preset)	SSR title page, <code>/api/r t c</code> signaling.

3. Repository map

Almost all game logic lives under `src/game/`. React is a thin chrome around one `SnowCraftGame` instance.

Path	Role
<code>src/components/game/SnowCraft.tsx</code>	React overlay: title, Easy/Hard, lobby, pause, HUD, QR.
<code>src/game/game.ts</code>	Orchestrator: screens, input, audio, net, loop.

<code>src/game/sim.ts</code>	Pure world step: throws, collisions, forts, bury.
<code>src/game/ai.ts</code>	Bot brains: idle / move / windup / dodge. Easy vs Hard.
<code>src/game/render.ts</code>	Canvas paint. Guest view is mirrored.
<code>src/game/net.ts</code>	Wire types, pack/apply pose & snapshot.
<code>src/game/versus-link.ts</code>	P2P + HTTP bus, seq, ping, clock offset.
<code>src/game/room-channel.ts</code>	SSE / poll fallback when WebRTC is down.
<code>src/lib/multiplayer/p2p.ts</code>	RTCPeerConnection mesh, ICE, data channels.
<code>src/lib/multiplayer/signaling.server.ts</code>	Room roster, ICE servers, TURN from env.
<code>src/routes/index.tsx</code>	Landing route (SSR title).
<code>src/routes/credits.tsx</code>	Licenses + tribute links.
<code>src/routes/api/rtc.ts</code>	Signaling HTTP endpoint.
<code>src/game/assets.ts / audio.ts</code>	Sprite hydrate (idle first, poses later) + SFX/BGM.

3.1 Layer cake

React chrome · SnowCraft.tsx, routes, credits

Buttons, lobby, pause — never the physics

SnowCraftGame · game.ts

Input, screens, hydrate, versus, audio unlock

Simulation · sim.ts + ai.ts

Fixed timestep. Hits, forts, bots. Authority lives here on the host.

Net · versus-link + p2p + /api/rtc

Poses, throws, hits, over. Signaling is not gameplay.

View · render.ts + sprites

Paint interpolated kids. Guest canvas is mirrored.

Data flows down: pointer → game.ts → sim (host) or local prediction (guest). Paint always reads the current GameState. The guest never decides a bury or a winner.

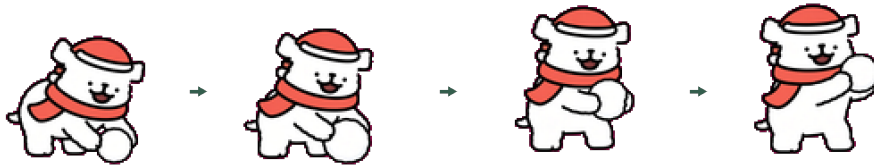
4. Screen flow





The only grey band we keep: 3-2-1 (here “PVP mode” + 3). Dogs may walk; packing is locked.

Title is React-only: no sprite preload, no AudioContext, so the first tap is live. Sprites load only after Easy/Hard, or after both PvP seats are in the room (packed handshake). Countdown lets dogs walk but not pack snow. Vs AI can return to title from pause. Vs Friend stays in the room on defeat so rematch does not need a new code.

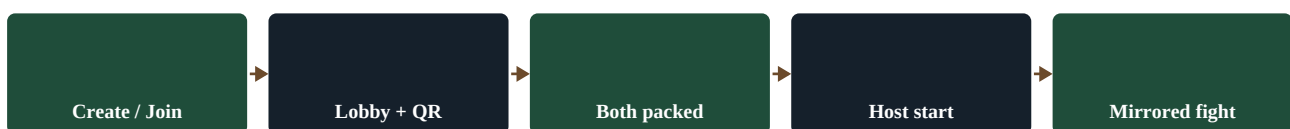


Pack-snow cooldown as a sprite strip (transparent). Four frames: crouch and wind a ball. Not in a fort.

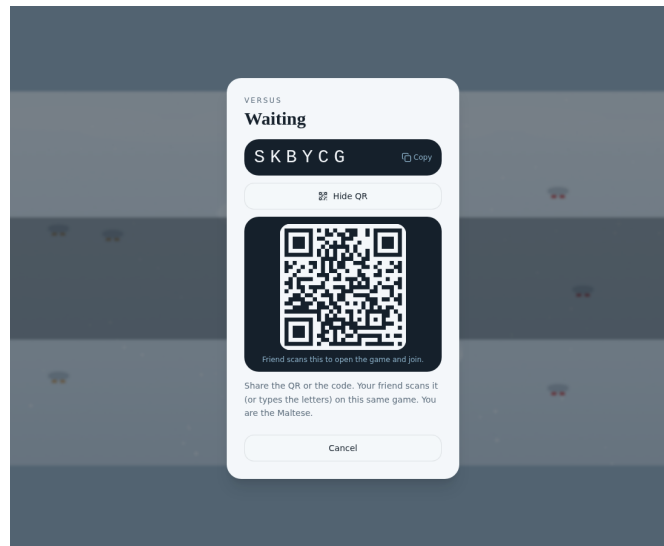


Throw cycle as a sprite strip: wind-up, release, follow-through, then the snowball. Guest predicts this locally.

4.1 Versus join



Host mints a 6-letter code. Guest types it or scans the QR. Neither side starts the heat until both have finished packing sprites. Host then sends **start**. Guest renders the yard flipped so the retrievers sit on the guest’s right.



Host invite: 6-letter code plus Show QR. Friend scans the code to open this same game and join.

5. Making P2P feel smooth

This is the expensive part of the project. A 1v1 snowball fight is sparse (a few dogs, occasional throws) but unforgiving: if a hit lands a beat late, it feels broken. The host’s machine runs the real simulation. The guest’s machine predicts, interpolates, and is corrected. There is no GGPO rollback and no dedicated sim server — on purpose (see 5.8). Below is every technique that is actually in the code, then a separate graveyard of attempts that did not stay.

5.1 The unfair clock

Host-authoritative P2P is uneven by default: the host is 0 ms from the sim, the guest is one round-trip away. LAN still feels lopsided. A game server would make *both* sides guests (fairer, not always faster). For a friend-code 1v1 we instead delay the host a little and predict on the guest.



Host: Maltese (white, right). Same room code in the HUD (Maltese).



Guest: mirrored. White Maltese now on the left; retrievers on the right. HUD says Retriever.

Seat	Without help	With current netcode
Host (Maltese)	Instant move, instant throw, instant hit.	Move still instant. Throw shows a ghost ball at once; real collision waits clamp(RTT/2, 30–80 ms).
Guest (retriever)	Waits a full snapshot (~70–220 ms) to see anything.	Own move/throw are local. Opponent pose interpolates 55–70 ms. Hit spark may paint early on high RTT; HP/SFX wait for host hit.

5.2 What actually goes over the wire

Early builds sent the whole yard 10–14 times a second (every kid, every ball, forts, phase). That is simple and laggy — see 5.8. The current split. Pose also copies onto the reliable channel because LAN WebRTC unordered often drops:

Message	Channel	Rate / when	Payload
pose	Unreliable + reliable copy	~14–20 Hz	id, x, y, vx, vy, facing, state, hp + live balls
allypose	Unreliable + reliable copy	Guest ~15–25 Hz, always	Guest-yard kids x,y (even Manual idle)
throw / allythrow	Reliable + HTTP copy	On release	id, origin, velocity, team, t0
hit	Reliable + HTTP copy	On damage	id, hp, heavy (bury). Only host.
snap	Unreliable + reliable copy	Every 1.2 s + start	Full field for recovery only

over / start / rematch / packed / bye	Reliable + unreliable + HTTP; no seq-drop	Events; over repeats 0.35 s	Winner (tiny over has no snapshot)
input	Move dual-sent; down/up reliable	Guest stick + throw	x, y, hold, vx, vy, at (timestamp)
ping / pong	Unreliable + reliable copy	2 s	t0, t1 → RTT + clock offset

5.3 Catalogue of smoothness work

Numbered as built, not as a wish list. This is the section that answers “what did you do so PvP does not feel like a slideshow”.

1. Host input delay

Host throws are queued by clamp(RTT/2, 30–80) ms before they enter the authoritative sim. Guest input already paid the network RTT, so both throws land in a similar window. Movement stays immediate so the dog still follows the finger.

2. Host ghost ball (local prediction)

On release the host spawns a ghost snowball: it flies and paints, but does not collide. When the delay elapses the ghost is killed and a real ball is committed + broadcast. The host sees an instant throw; fairness stays in the delayed hit test.

3. Guest local move & throw

Guest dogs follow the pointer with no round-trip. Guest throws spawn local balls immediately and also send input.at to the host. Pose packets do not overwrite the grabbed dog.

4. RTT-tiered hit juice (cosmetic only)

HP, bury, bark, and winner are host-only. Guest paint depends on smoothed RTT, decided live (not at room setup): <55 ms wait for host; 55–120 ms spark only; >120 ms spark + light hurt pose that rolls back. Upgrade is instant; downgrade waits 2 s. Aim uses the latest pose plus half-RTT, not the interpolated ghost.

5. Remote interpolation (guest only, paint only)

Guest paints host Maltese 55–70 ms in the past, lerped between pose samples. The grabbed local dog is not interpolated. Host draws guest dogs at live sim x,y (no second delay). This replaced vx·lead extrapolation.

6. Clock offset

Ping carries t0; pong echoes t0 and t1. Each side estimates peerTime – localTime, smoothed. Guest uses hostNow ≈ now + offset when applying poses. Guest inputs carry at; the host maps them into its delay buffer instead of applying wall-clock-now.

7. Keyframes do not clobber balls

The 1.2 s snapshot repairs kids and HP. Local and ghost balls are kept unless a host ball from the same thrower is already nearby. Predicted throws no longer pop out of existence.

8. Two data channels + always-copy poses

Unordered ‘state’ channel plus ordered ‘event’ channel. Throws, hits, over stay reliable + HTTP. Poses, allypose, input-move, and snaps also copy onto reliable: LAN/iOS unordered SCTP silently drops, and dogs used to freeze until dragged. Separate seq spaces (lastIn vs lastPose) so a pose cannot eat rematch.

9. Outcome is not a snapshot

A fat WireState over packet was easy to drop. Now a tiny { winner } is sent as well, resent every 0.35 s, and poses keep advertising phase won/lost after the heat. Guest can infer the result three ways. Rematch start is recovered from a fight pose if both players already voted yes.

10. Packed handshake

The match does not start until both browsers have the pose sprites. Prevents one side fighting blobs while the other has dogs, which felt like desync.

11. ICE / TURN / relay HUD

STUN first; ExpressTURN from server env (never committed). ICE restart if stuck; fall back to SSE poll. HUD shows direct vs relay, RTT, fps so we can tell lag from art.

12. Guest-local presentation

Idle fidget, walk cycle, aim line, particles, SFX, and 3-2-1 ticks run locally. They are not networked. Traffic stays tiny; the yard still looks alive.

13. Ally bots on the guest

Defend / Attack step locally on the guest; allythrow + allypose go up. Guest always sends yard poses, even in Manual, after clamp-to-screen. Guest presentation ticks packT and cooldown so bots do not freeze after the first throw.

14. Live clock, not room-setup lag class

RTT EMA (0.8/0.2) drives host delay, interpolation window, and hit-FX tier. A lobby ping is stale: ICE can flip STUN→TURN mid-match. Upgrade hit-tier immediately; hold 2 s to drop.

15. Throw timestamps (t0) on both seats

Host, guest, and ally throws carry t0 (plus guest input.at). The host maps peer time through clockOffset and either queues a future throw or catch-up-simulates a late one. Charge seconds are ignored in Versus.

16. Host catch-up (projectile, not hitscan rewind)

If a guest throw arrives up to ~200 ms late, the host looks up the thrower in pose history, spawns the ball there, and steps that one ball forward against historical kid positions. This is not CS lag compensation (no rewind of the whole yard for a hitscan). Snowballs fly; we only fix when they were born.

17. Versus throw profile

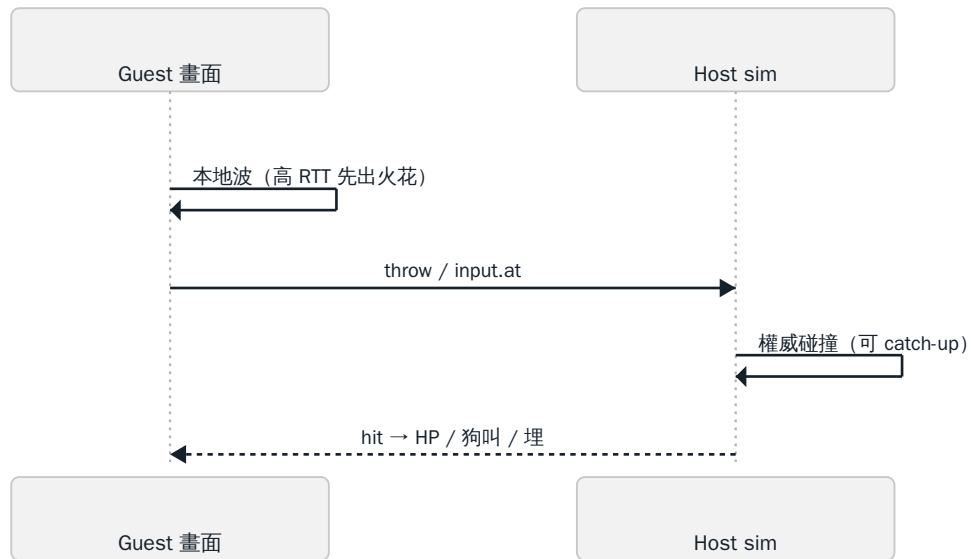
PvP: auto-aim nearest foe; fixed range 520 and speed 440; tap and hold fly the same. Pack remains 0.92 s. Vs AI keeps tap-near / hold-far and Hard's 2× ball speed. Guest replicas of opponent balls use the same 520 range (not 2400).

5.4 Throw path (sequence)

Host runs the real collision. Guest only paints: local ball, optional spark, interpolated Maltese. HP changes on hit; the heat ends on over. A wrong spark rolls back; the whole field does not.

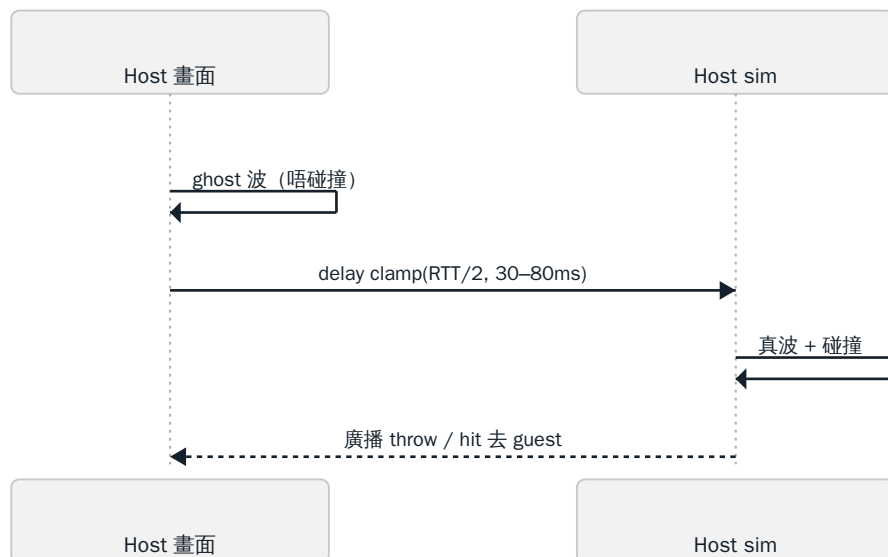
Guest throw (how it feels vs how it scores)

Guest 出手：畫面可以即時，HP 同聲效等 host。



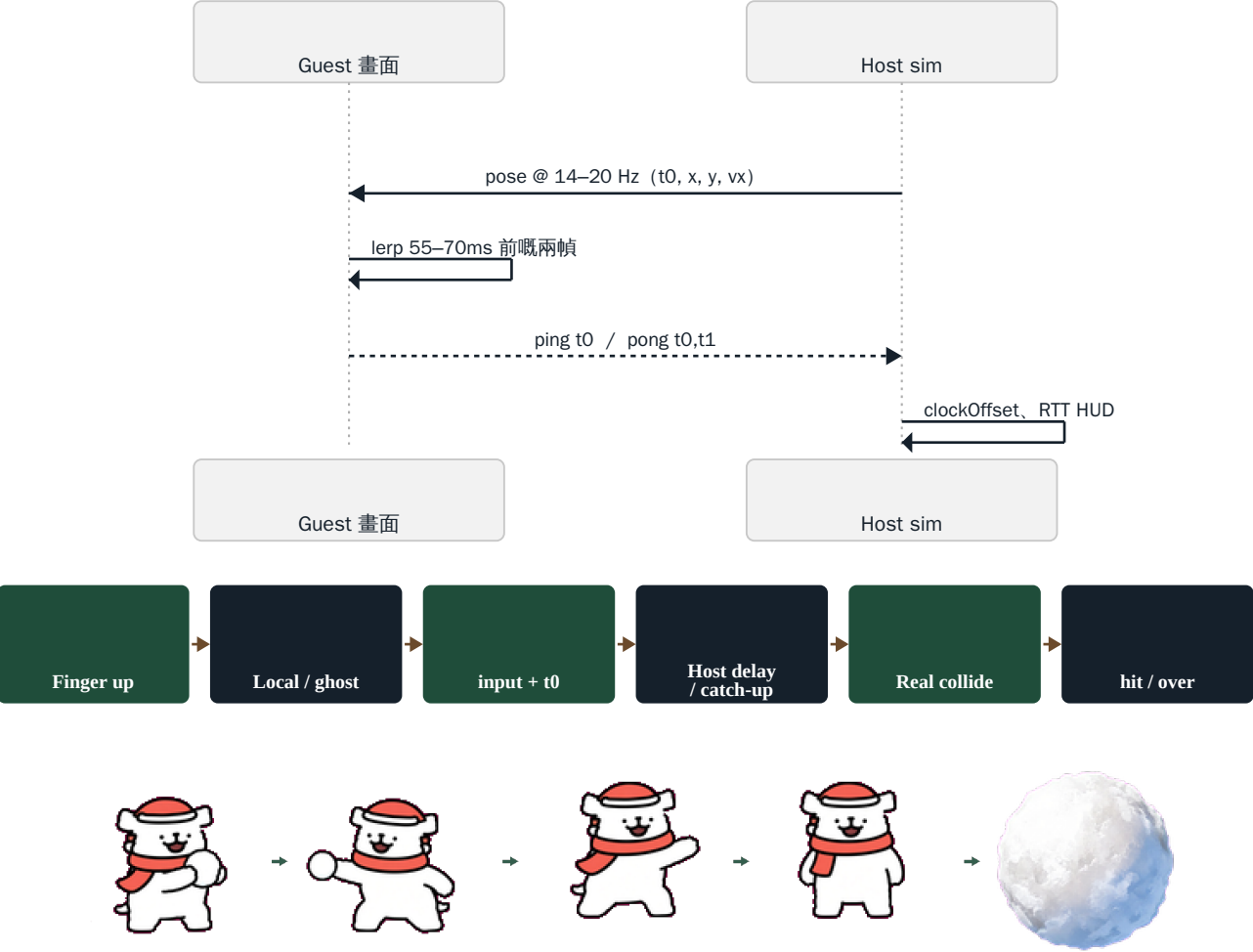
Host throw (ghost, then delayed real ball)

Host 出手：即時畫面，命中刻意遲半程 RTT。



Clock + interpolation

對手狗插值；自己拖住嗰隻跟手指，唔入過去。



Throw strip: local art first, host collision later. Versus always this range.

5.5 Why not a sim server

A server would equalize latency (both sides ~RTT to Tokyo/Singapore) and help anti-cheat. It would not make LAN faster, and it would add 24/7 ops. For a 6-dog yard, pose+event+prediction is the right cost curve. Revisit a server if we add public matchmaking or need to stop a host from lying about HP.

5.6 Versus combat recipe

Versus is not the same sport as Vs AI. Charge was a local timer and did not cost RTT, but it split tap-near / hold-far and made guest/host hold seconds a possible disagreement. PvP now: auto-aim the nearest enemy for **direction only**; **range 520** and **speed 440** every shot; pack still 0.92 s. A tap and a long hold fly the same. Hard’s 2× ball speed is not used in Versus — it would make delay a large fraction of flight time.

Knob	Vs AI	Versus
Aim	Nearest (optional scatter on Hard)	Nearest foe, both seats
Charge	Tap near, 1.2 s hold = full field	None — tap = hold
Range / speed	58–820 px / 360–500 (Hard ×2)	520 / 440, both seats + replicas

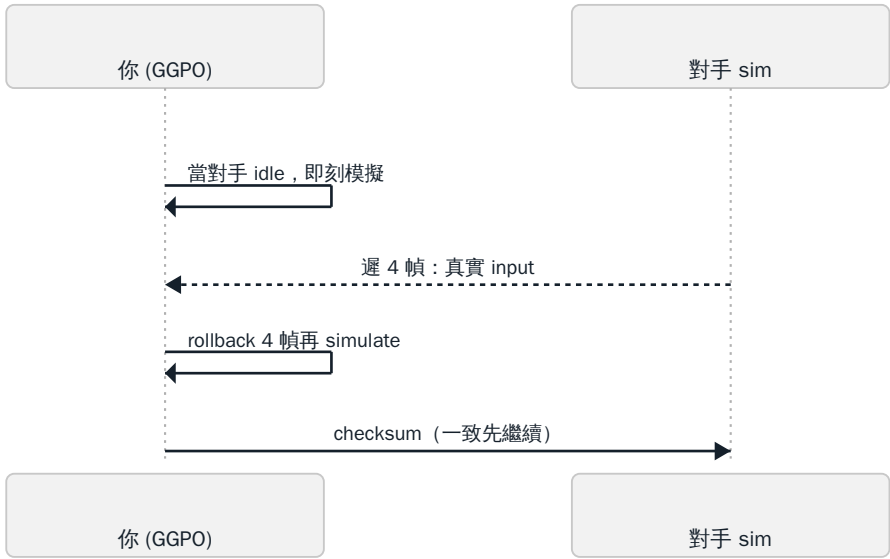
Pack	0.92 s	0.92 s (reload is the cadence)
------	--------	--------------------------------

5.7 How this sits vs GGPO, RTS, CS

We borrowed CS’s idea (predict yourself, authority elsewhere) and RTS’s idea (don’t snapshot the whole army). We did not take fighting-game rollback or RTS lockstep.

Family	Sends	Authority	We took
RTS lockstep (AoE, SC)	Commands only	Everyone’s same sim	Tiny events, not 2000-unit snapshots. Not lockstep — we need instant drag.
CS / Source FPS	Input + server snapshots	Dedicated server; rewind hitscan	Local move, interp remotes. Snowballs are projectiles: catch-up spawn, not hitscan rewind.
GGPO / rollback	Per-frame input	Both sims, rewind on mismatch	Not used. Sim has Math.random and wall clocks. Flash rollback only.
This game	Pose 14–20 Hz + throw/hit/over	Host browser	Listen-server CS + RTS-sized payloads + host delay to fake fairness.

對照：GGPO 兩邊都係真 sim，錯就倒帶成場。雪仗唔做呢步。



5.8 Failed and unused netcode

This section is the graveyard: ideas we shipped, then ripped out, or never merged. They are not in the live stack. Keeping them here so the next pass does not rebuild the same trap.

Attempt	What happened	Kept instead
Full-yard snapshot 12–20 Hz as the live stream	Simple. Fat. Guest felt 70–220 ms behind even on LAN. Balls popped when a snap arrived late.	Thin pose + event throws/hits. Snap only as a 1.2 s keyframe that does not clobber local balls.

HTTP poll ~40 ms as the primary pipe	Worked through bad NATs. Added a full RTT of jitter and serialized the yard through JSON poll.	WebRTC first. HTTP/SSE is a belt for join, ICE, and rare event copies when the DC is down.
Unreliable-only poses (unordered SCTP)	LAN/iOS often dropped the unordered channel with no error. Dogs froze until that dog was dragged (the drag used a reliable input).	Every pose / allypose / snap is copied onto the reliable channel. Unreliable stays for freshness.
One sequence space for poses and events	A high pose seq made the guest drop rematch / over / start as 'stale'.	lastIn for events, lastPose for pose-class. Outcome packets ignore seq drop.
Guest always-on hit prediction (HP/SFX)	Felt snappy, then '■■■■■■': spark + bark on a miss because the guest tested against a lagged dog.	Host hit is the only HP/SFX. Cosmetic spark is RTT-tiered against latest pose + half-RTT, never viewX.
Predict hits on interpolated viewX	The paint pose is 55–70 ms in the past. Testing the ball against that ghost false-positives more.	Hit juice uses sim x,y (or pose history velocity), not the interpolated sprite.
Wait ~220 ms before reconciling balls	Ghost balls hung on the guest; real balls appeared late; clashes looked random.	Guest flies every ball now. Unmatched local host-throws prune after a short hold, not 220 ms.
vx·lead dead reckoning	Extrapolating opponent $x \pm vx \cdot rtt$ fought host delay and made dogs skate through forts.	Interpolation in the past on guest only. Host draws guest dogs live.
Host interpolating guest dogs from poseHist	Host already has guest input. Adding 70 ms of lerp made the host see their own applied moves late.	Only the guest interpolates remote (Maltese). Host viewX = sim x.
allypose only while Defend/Attack	Manual guest still clamp-to-screen locally. Host never heard those x,y until that dog was dragged.	Guest always streams living retriever poses, grabbed dog excluded (that one uses input-move).
Guest skipping cooldown / pack ticks	Guest does not run stepSim. After the first ally throw, cooldown stuck > 0, windup never finished. Bots stood still.	stepPresentation now ticks cooldown, packT, stun, freeze. Windup falls back to idle if blocked.
Fat WireState over packet	Easy to drop; guest and host could show different winners.	Tiny { winner } on reliable + HTTP, resent 0.35 s, plus pose.phase won/lost.
Charge-based PvP throws	Tap-near / hold-far disagreed across seats (different hold seconds, different RTT). Unfair on WAN.	Versus: auto-aim direction, fixed range 520, speed 440, pack 0.92 s. Charge remains Vs AI only.
GGPO / full rollback	Needs a deterministic sim. Ours uses Math.random, wall clocks, sprite timers. Six dogs + balls is a lot to rewind every frame.	Listen-server. Cosmetic rollback of a spark only. See 5.7.
Dedicated sim server	Fairer RTT (both sides guests) and anti-cheat. 24/7 ops. LAN would get slower, not faster.	Host browser + signaling + TURN. Revisit if public matchmaking or ranked anti-cheat.
Binary protobuf / bit-packed poses	JSON pose is already ~a few hundred bytes at 20 Hz. Extra schema tax for little gain.	JSON envelopes. Revisit if we add 4-player or mobile data caps.
TURN credentials in the repo	Would leak ExpressTURN on GitHub.	Server env only. Public .env.example stays empty.

Rule of thumb we landed on: **host owns score, guest owns feel, never trust a dropped UDP as truth, never predict HP**. Everything in the table above broke one of those.

6. Vs AI (for contrast)

Easy: original pacing, nearest-target bots, forts are cover only, Maltese revive next heat. Hard: bot move $\times 3$, smarter dodge, forts have 10 HP, no Maltese revive, Maltese move $\times 2$ and both teams' snowballs $\times 2$. Same sim as PvP; no net. Difficulty is chosen on a gate after Play vs AI, which is also when sprites load (progress bar).

7. Loading strategy

SSR paints a Loading spinner until React hydrates, then the title card. Play vs AI / Friend are real buttons only after hydrate. hydrateCore (idle dogs + fort) and hydrateRest (throw/hurt/walk/...) run after a mode is chosen. Unused placeholder characters were removed so the landing wait is not packing dead art.

8. Trust and secrets

TURN username/credential live in server env / local .env, never in git. The public repo only has empty TURN_USERNAME / TURN_CREDENTIAL in .env.example. P2P gameplay is host-honest: a cheating host can still lie. Fine for friends; not for ranked.

9. Credits (short)

Game: Nicholson NY, SnowCraft, 1998. Characters: moonlab / Line Puppy (fan illustration). Fight feel referenced jeffreywilbur/snowcraftjs. Libraries (MIT / Apache-2.0 / ISC / OFL) are listed in-app at /credits.

End of briefing. Source of truth is the beta branch of the GitHub repo; this PDF describes the live P2P stack as of August 2026 (pose dual-send, RTT-tiered juice, guest ally timers). Section 5.8 lists netcode that was tried and removed.